

Documentation Technique d'Implémentation — Rébus & Indices (Unreal Engine C++)

Version mise à jour : Support Grilles Impaires, Échelle Métrique & PathFinder Annexes

Guide **pas à pas** pour implémenter, dans le projet `Cube` (UE 5.6 → 5.8, C++), le système d'indices/rébus conçu dans `Rebus_Documentation.md`, au-dessus du solveur (`Solveur_Documentation.md`). Ce document constitue la référence technique unifiée ("Technical Game Design") synchronisée bit-à-bit avec l'outil de validation Blender Python.

0. Périmètre, hypothèses et conventions

Point	Décision
Version	UE 5.8 (compatible 5.6/5.7), Enhanced Input obligatoire
Module	<code>Cube</code> (existant) — dépendances : <code>UMG</code> , <code>EnhancedInput</code> , <code>Niagara</code> , <code>OnlineSubsystem*</code> , <code>HTTP</code>
Préfixes projet	C++ cœur : <code>Cube*</code> (<code>UCubeGenerator</code> , <code>UCubeSolver</code> , <code>FCubeManifest</code>) · <code>GameMode</code> <code>*GameMode</code> / <code>GM_</code> · <code>DA_</code> data assets · <code>BP_</code> acteurs · <code>BPI_</code> interfaces · <code>UI_</code> widgets
Classes réutilisées	<code>UCubeGameInstance</code> , <code>AMenuGameMode</code> , <code>ABPC_PlayerController</code> , <code>APS_PlayerStateCustom</code> , <code>UCubeGenerator</code> , <code>UCubeSolver</code> , <code>FCubeMath</code> , <code>ECubeDirection</code>
Réseau	Local pour les paliers Facile/Impossible ; serveur-autoritaire pour le palier secret Championnat du Monde (§17-18).
Performance	Volume massif en <code>HISM</code> (<code>ACubeGrid</code>) ; acteurs riches <code>BP_CUBE</code> poolés et streamés autour du joueur (§15).
Échelle métrique	<code>RoomSize</code> (Pitch) paramétrable (défaut 500 cm / 5m ou 1000 cm / 10m) centré en X/Y, posé au sol en Z.
Grilles supportées	Toutes tailles $N \in [10, 50]$ gérées sans singularité mathématique, qu'elles soient paires ou impaires (ex. 13, 27, 32).

Répartition des responsabilités (Gameplay Framework) :

Besoin	Classe	Portée
Choix menu (mode, seed) persistant	UCubeGameInstance	Tout le cycle du process
Règles : génération (Passe A+B), résolution, sécurité, victoire/mort	AGM_CubeGameMode	Serveur
État partagé répliqué (seed, N, pitch, phase)	AGS_CubeGameState	Tous clients
Progrès joueur (index chemin, morts, clés, checkpoint)	APS_PlayerStateCustom	Répliqué, survit au respawn
Input, HUD, commandes de déplacement	ABPC_PlayerController	Serveur + Client owner
Pawn joueur, coordonnées discrètes de cellule	ACubeCharacter	Répliqué
Salle physique : meshes, affichage route, dalle mathématique	ACubeRoom → BP_CUBE	Spawné à la demande
Panneau-formule mural interactif	ACubeMathPath → BP_Math_Path	Enfant de la salle
Cœur algorithmique pur (Champ mortel, Formules, PathFinder Annexes)	UCubeRebusLibrary	Statique, Engine-agnostic
Streaming / pooling des salles autour du joueur	UCubeRoomStreamer	Sous-système World
Données de calibrage par difficulté	DA_CubeDifficulty	Data Asset
Interfaces d'interaction et de notification	ICubeRoom , ICubeHint , ICubeInteractable	Interfaces C++

1. Types partagés — CubeRebusTypes.h

Nouveau header dans `Public/`. Regroupe enums et structures Blueprint-exposées :

```

#pragma once

#include "CoreMinimal.h"
#include "CubeCore.h"                // ECubeDirection, FCubeCoordinate, FCubeMath
#include "CubeRebusTypes.generated.h"

/** Palier de difficulté */
UENUM(BlueprintType)
enum class ECubeDifficulty : uint8
{
    Facile,                // T0 : route RLE complète, aucune mort, boussole active
    Impossible,            // T1 : grilles 10..50, champ mortel local (Rho), formules modulaires
    WorldChampionship     // T2 : secret serveur-autoritaire, formules masquées en RAM
};

/** État logique d'une pièce (+ conventions de couleurs DEBUG) */
UENUM(BlueprintType)
enum class ECubeCellState : uint8
{
    SafePath,              // Sur le chemin critique du solveur      – Debug : Turquoise (0.0, 0.85, 0.85)
    SafeShortcut,          // Raccourci sain reliant deux pas du chemin – Debug : Vert (0.1, 0.85, 0.1)
    Lethal,                // Piège mortel au contact                  – Debug : Rouge (0.85, 0.1, 0.1)
    Inactive,              // Roche/Mur ou cul-de-sac non reliant      – Debug : Gris sombre (0.05, 0.05, 0.06)
    Start,                 // Entrée
    Exit,                  // Sortie
    Key,                   // Emplacement clé
    Gate                   // Porte verrouillée
};

/** Un terme RLE : direction + longueur (ex. +Y x2 → "Y+(2)") */
USTRUCT(BlueprintType)
struct CUBE_API FCubeRouteToken
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly) ECubeDirection Dir = ECubeDirection::None;
    UPROPERTY(BlueprintReadOnly) int32 Run = 0;
};

/**
 * Formule affichée sur le mur. Le résultat sénéaire d in {0..5} désigne la
 * direction sûre (index de ECubeDirection - 1).
 */
USTRUCT(BlueprintType)
struct CUBE_API FCubeFormula
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly) int32 Tier = 1;
    UPROPERTY(BlueprintReadOnly) int32 A1 = 0;
    UPROPERTY(BlueprintReadOnly) int32 A2 = 0;
    UPROPERTY(BlueprintReadOnly) int32 A3 = 0;
    UPROPERTY(BlueprintReadOnly) int32 A4 = 0;    // Terme libre calibré
    UPROPERTY(BlueprintReadOnly) int32 Mod = 6;

    // Non exposé au client en compétition ; validateur serveur
    int32 Answer = 0;
    bool bEnableDecoy = false;
    UPROPERTY(BlueprintReadOnly) ECubeDirection Decoy = ECubeDirection::None;
};

/** Configuration complète d'une salle pour BP_CUBE */

```

```

USTRUCT(BlueprintType)
struct CUBE_API FCubeRoomData
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly) int32 RoomID = -1;
    UPROPERTY(BlueprintReadOnly) FCubeCoordinate Cell;
    UPROPERTY(BlueprintReadOnly) ECubeCellState State = ECubeCellState::Inactive;
    UPROPERTY(BlueprintReadOnly) int32 PathIndex = -1; // Rang sur le chemin (-1 si hors chemin)
    UPROPERTY(BlueprintReadOnly) bool bHasFormula = false;
    UPROPERTY(BlueprintReadOnly) FCubeFormula Formula;
};

```

2. Hash déterministe canonique — CubeHash.h

Pour garantir la stricte parité **bit-à-bit** entre Python (Blender) et C++ (Unreal), on utilise le polynôme **CRC32 IEEE 802.3** (`zlib.crc32`) combiné avec le mélangeur Boost :

```

#pragma once
#include "CoreMinimal.h"

namespace CubeHash
{
    inline uint32 Crc32(const uint8* Data, int32 Len)
    {
        static uint32 Table[256];
        static bool bInit = false;
        if (!bInit)
        {
            for (uint32 i = 0; i < 256; ++i)
            {
                uint32 c = i;
                for (int k = 0; k < 8; ++k)
                    c = (c & 1) ? (0xEDB88320u ^ (c >> 1)) : (c >> 1);
                Table[i] = c;
            }
            bInit = true;
        }
        uint32 c = 0xFFFFFFFFu;
        for (int32 i = 0; i < Len; ++i)
            c = Table[(c ^ Data[i]) & 0xFFu] ^ (c >> 8);
        return c ^ 0xFFFFFFFFu;
    }

    inline uint32 Crc32Str(const FString& S)
    {
        FTCHARToUTF8 Utf8(*S);
        return Crc32(reinterpret_cast<const uint8*>(Utf8.Get()), Utf8.Length());
    }

    // hash_combine(a, b) = (a ^ (b + 0x9E3779B9 + (a << 6) + (a >> 2))) mod 2^32
    inline uint32 Combine(uint32 A, uint32 B)
    {
        return A ^ (B + 0x9E3779B9u + (A << 6) + (A >> 2));
    }
}

```

3. Cœur algorithmique — UCubeRebusLibrary

Bibliothèque de fonctions pures moteur/test : 1. Calcul du champ mortel déterministe (`IsLethal`). 2. Détection stricte des raccourcis viables (`ComputeSafeShortcuts` / `PathFinder_Annexes`). 3. Générateur et solveur de formules modulaires (`MakeFormula` , `EvaluateFormula`).

3.1 Header — Public/CubeRebusLibrary.h

```
#pragma once
#include "CoreMinimal.h"
#include "Kismet/BlueprintFunctionLibrary.h"
#include "CubeRebusTypes.h"
#include "CubeLogicTypes.h"
#include "CubeRebusLibrary.generated.h"

UCLASS()
class CUBE_API UCubeRebusLibrary : public UBlueprintFunctionLibrary
{
    GENERATED_BODY()
public:
    /** Compression de route en tokens RLE */
    UFUNCTION(BlueprintCallable, Category="Cube|Rebus")
    static TArray<FCubeRouteToken> RouteToRLE(const TArray<int32>& PathRoomIDs, int32 GridSize);

    /** Évalue si une cellule est un piège mortel selon Seed + Rho */
    static bool IsLethal(const FString& Seed, int32 X, int32 Y, int32 Z,
        float Rho, const TSet<int32>& SafeSet, int32 GridSize);

    UFUNCTION(BlueprintCallable, Category="Cube|Rebus")
    static bool IsLethalCell(const FString& Seed, FCubeCoordinate Cell,
        float Rho, int32 GridSize, const TSet<int32>& SafeSet);

    /**
     * PathFinder_Annexes : Identifie STRICTEMENT les cellules qui forment un véritable raccourci.
     * Une cellule non-mortelle n'est un SafeShortcut que si elle appartient à une composante
     * reliant au moins 2 points distincts et non-adjacents du chemin critique.
     */
    UFUNCTION(BlueprintCallable, Category="Cube|Rebus")
    static TSet<int32> ComputeSafeShortcuts(const FString& Seed, const TArray<int32>& PathRoomIDs,
        int32 GridSize, float Rho);

    /** Calibre la formule de porte modulaire pour que le résultat pointe sur la direction sûre */
    static FCubeFormula MakeFormula(const FString& Seed, int32 PathIndex,
        FCubeCoordinate Pos, int32 SafeDirIndex,
        int32 Tier, bool bEnableDecoy);

    UFUNCTION(BlueprintCallable, Category="Cube|Rebus")
    static int32 EvaluateFormula(const FCubeFormula& F, FCubeCoordinate Pos);

    UFUNCTION(BlueprintCallable, Category="Cube|Rebus")
    static ECubeDirection IndexToDirection(int32 Index);
    static int32 DirectionToIndex(ECubeDirection Dir);
};
```

3.2 CPP — Private/CubeRebusLibrary.cpp

```
#include "CubeRebusLibrary.h"
#include "CubeHash.h"

static FORCEINLINE int32 PosMod(int32 A, int32 M) { return ((A % M) + M) % M; }

bool UCubeRebusLibrary::IsLethal(const FString& Seed, int32 X, int32 Y, int32 Z,
                                float Rho, const TSet<int32>& SafeSet, int32 GridSize)
{
    const int32 RoomID = FCubeMath::EncodeCoord(FCubeCoordinate(X, Y, Z), GridSize, GridSize);
    if (SafeSet.Contains(RoomID)) return false; // Le chemin du solveur reste toujours sain

    uint32 H = CubeHash::Crc32Str(Seed);
    H = CubeHash::Combine(H, CubeHash::Crc32Str(TEXT("TRAP")));
    const uint32 Spatial = ((uint32)(X * 73856093)) ^ ((uint32)(Y * 19349663)) ^ ((uint32)(Z * 83492791));
    H = CubeHash::Combine(H, Spatial);
    return (H % 10000u) < (uint32)FCubeMath::FloorToInt(Rho * 10000.0f);
}

TSet<int32> UCubeRebusLibrary::ComputeSafeShortcuts(const FString& Seed, const TArray<int32>& PathRoomIDs,
                                                    int32 GridSize, float Rho)
{
    TSet<int32> ShortcutCells;
    if (PathRoomIDs.Num() == 0 || Rho <= 0.0f) return ShortcutCells;

    const int32 N = GridSize;
    const TSet<int32> PathSet(PathRoomIDs);
    TMap<int32, int32> FirstIdx;
    for (int32 i = 0; i < PathRoomIDs.Num(); ++i)
    {
        if (!FirstIdx.Contains(PathRoomIDs[i]))
            FirstIdx.Add(PathRoomIDs[i], i);
    }

    // 1. Détermination des candidats non-mortels hors chemin
    TSet<int32> CandidateSet;
    for (int32 Z = 0; Z < N; ++Z)
    {
        for (int32 Y = 0; Y < N; ++Y)
        {
            for (int32 X = 0; X < N; ++X)
            {
                const int32 ID = FCubeMath::EncodeCoord(FCubeCoordinate(X, Y, Z), N, N);
                if (!PathSet.Contains(ID) && !IsLethal(Seed, X, Y, Z, Rho, PathSet, N))
                {
                    CandidateSet.Add(ID);
                }
            }
        }
    }

    auto GetNeighbors = [N](int32 ID) -> TArray<int32> {
        FCubeCoordinate C = FCubeMath::DecodeCoord(ID, N, N);
        TArray<int32> Res;
        const int32 D[6][3] = {{1,0,0},{-1,0,0},{0,1,0},{0,-1,0},{0,0,1},{0,0,-1}};
        for (int32 k = 0; k < 6; ++k)
        {
            int32 nx = C.X + D[k][0], ny = C.Y + D[k][1], nz = C.Z + D[k][2];
            if (nx >= 0 && nx < N && ny >= 0 && ny < N && nz >= 0 && nz < N)
                Res.Add(FCubeMath::EncodeCoord(FCubeCoordinate(nx, ny, nz), N, N));
        }
    }
```

```

    return Res;
};

// 2. Exploration par composantes connexes et ponts BFS
TSet<int32> Visited;
for (int32 StartID : CandidateSet)
{
    if (Visited.Contains(StartID)) continue;

    TArray<int32> Component;
    TQueue<int32> Queue;
    Visited.Add(StartID);
    Queue.Enqueue(StartID);

    TSet<int32> TouchPoints;
    while (!Queue.IsEmpty())
    {
        int32 Curr;
        Queue.Dequeue(Curr);
        Component.Add(Curr);

        for (int32 Nb : GetNeighbors(Curr))
        {
            if (PathSet.Contains(Nb))
            {
                TouchPoints.Add(Nb);
            }
            else if (CandidateSet.Contains(Nb) && !Visited.Contains(Nb))
            {
                Visited.Add(Nb);
                Queue.Enqueue(Nb);
            }
        }
    }

    if (TouchPoints.Num() >= 2)
    {
        TArray<int32> SortedTouch = TouchPoints.Array();
        SortedTouch.Sort([&FirstIdx](int32 A, int32 B) { return FirstIdx[A] < FirstIdx[B]; });

        const TSet<int32> CompSet(Component);

        for (int32 i = 0; i < SortedTouch.Num(); ++i)
        {
            for (int32 j = i + 1; j < SortedTouch.Num(); ++j)
            {
                int32 U = SortedTouch[i];
                int32 V = SortedTouch[j];
                if (FirstIdx[V] - FirstIdx[U] >= 2)
                {
                    // BFS entre U et V à travers CompSet
                    TQueue<int32> QPath;
                    TSet<int32> QVis;
                    TMap<int32, int32> Parent;
                    QPath.Enqueue(U);
                    QVis.Add(U);
                    bool bFound = false;

                    while (!QPath.IsEmpty() && !bFound)
                    {
                        int32 Curr;
                        QPath.Dequeue(Curr);
                        for (int32 Nb : GetNeighbors(Curr))

```

```

        {
            if (Nb == V)
            {
                Parent.Add(Nb, Curr);
                bFound = true;
                break;
            }
            if (CompSet.Contains(Nb) && !QVis.Contains(Nb))
            {
                QVis.Add(Nb);
                Parent.Add(Nb, Curr);
                QPath.Enqueue(Nb);
            }
        }
    }

    if (bFound)
    {
        int32 Trace = Parent[V];
        while (Trace != U)
        {
            ShortcutCells.Add(Trace);
            Trace = Parent[Trace];
        }
    }
}

return ShortcutCells;
}

FCubeFormula UCubeRebusLibrary::MakeFormula(const FString& Seed, int32 PathIndex,
                                             FCubeCoordinate Pos, int32 SafeDirIndex,
                                             int32 Tier, bool bEnableDecoy)
{
    uint32 Hs = CubeHash::Combine(CubeHash::Crc32Str(Seed), (uint32)PathIndex);
    FCubeFormula F;
    F.Tier = Tier;
    F.Mod = 6;
    F.A1 = 1 + (Hs % 3);
    F.A2 = 1 + ((Hs >> 3) % 3);
    F.A3 = 1 + ((Hs >> 6) % 3);

    const int32 Base = F.A1 * Pos.X + F.A2 * Pos.Y + F.A3 * Pos.Z;
    F.A4 = PosMod(SafeDirIndex - Base, 6);
    F.Answer = SafeDirIndex;
    F.bEnableDecoy = bEnableDecoy;
    return F;
}

int32 UCubeRebusLibrary::EvaluateFormula(const FCubeFormula& F, FCubeCoordinate Pos)
{
    return PosMod(F.A1 * Pos.X + F.A2 * Pos.Y + F.A3 * Pos.Z + F.A4, F.Mod);
}

```

4. Spécifications du Générateur C++ (UCubeGenerator)

Pour garantir qu'aucun blocage ou freeze n'intervienne (identique au fix apporté au code Blender Python) :

4.1 Carving sur grilles impaires (\$N\$ impair)

1. **Borne paire intérieure** : L'algorithme de bonds par pas de 2 impose une borne paire : $\text{hi} = (N - 2) \setminus \& \setminus \sim 1$
2. **Jonction de sortie** : Avant d'atteindre la face opposée $(N-1, \text{ey}, \text{ez})$, le générateur insère des connecteurs linéaires tant que $cx < N - 2$: `cpp while (cx < N - 2) { cx += 1; FullPath.Add(FCubeCoordinate(cx, cy, cz)); } FullPath.Add(FCubeCoordinate(N - 1, ey, ez));`

4.2 Garantie de Solvabilité (Boucle Déterministe)

Le générateur implémente une boucle d'essai incrémentale sur sous-seed pour éliminer tout risque de seed orpheline :

```
int32 Attempt = 0;
while (true)
{
    Manifest = GenerateManifestAttempt(Seed, Attempt, N, PathLen, Verticality, NKeys);
    if (Manifest.bValid && Solver->SolveManifest(Manifest).bSolved)
    {
        break; // Succès garanti
    }
    Attempt++;
}
```

5. Gestion de l'Échelle et des Instances (RoomSize / HISM)

Le positionnement dans l'espace monde Unreal respecte la formule de centrage : $P_{\text{world}} = \left(X - \frac{N - 1}{2} \right) \cdot \text{RoomSize} \cdot \vec{u}_X + \left(Y - \frac{N - 1}{2} \right) \cdot \text{RoomSize} \cdot \vec{u}_Y + \left(Z + 0.5 \right) \cdot \text{RoomSize} \cdot \vec{u}_Z$

- ACubeGrid instancie les coques de pièces via un UHierarchicalInstancedStaticMeshComponent (HISM).
- Lors de l'assignation d'une salle custom (Custom Room), les instances utilisent les positions des sommets calculées, assurant une fluidité maximale à 60 FPS sur des volumes de 64^3 (262 144 cubes).

6. GameMode Serveur & Gestion de la Difficulté

6.1 AGM_CubeGameMode.h

```
#pragma once
#include "CoreMinimal.h"
#include "GameFramework/GameModeBase.h"
#include "CubeRebusTypes.h"
#include "CubeLogicTypes.h"
#include "GM_CubeGameMode.generated.h"

class UDA_CubeDifficulty;
class UCubeRoomStreamer;

UCLASS()
class CUBE_API AGM_CubeGameMode : public AGameModeBase
{
    GENERATED_BODY()
public:
    AGM_CubeGameMode();
    virtual void BeginPlay() override;

    UPROPERTY(EditDefaultsOnly, Category="Cube|Difficulty")
    TMap<ECubeDifficulty, TObjectPtr<UDA_CubeDifficulty>> DifficultyTable;

    UPROPERTY(EditDefaultsOnly, Category="Cube|Geometry")
    float RoomSize = 500.0f; // Espacement métrique (cm)

    UPROPERTY(EditDefaultsOnly, Category="Cube|PathFinder")
    bool bEnableShortcuts = true; // Active l'évaluation des SafeShortcuts

    void NotifyRoomEntered(class ABPC_PlayerController* PC, FCubeCoordinate Cell);

protected:
    void BuildLevel();
    int32 PickGridSize(const FString& Seed) const;

    UPROPERTY() FCubeManifest Manifest;
    TArray<int32> PathRoomIDs;
    TMap<int32, int32> PathIndexByRoom;
    TSet<int32> SafeSet;
    TSet<int32> ShortcutSet;
    ECubeDifficulty Difficulty = ECubeDifficulty::Facile;

    UPROPERTY() TObjectPtr<UDA_CubeDifficulty> Config;
    UPROPERTY() TObjectPtr<UCubeRoomStreamer> Streamer;
};
```

7. Plan de Test et Validation Automatisée

Le module de test d'automatisation C++ (FAutomationTestBase) valide :

1. **Parité bit-à-bit Python↔C++** :
2. Exécution de 10 000 cellules comparant IsLethal entre Blender et UE5 (\$\Delta = 0\$).
3. Validation de l'évaluation modulaire de MakeFormula et EvaluateFormula .
4. **Validation PathFinder_Annexes** :

5. Vérification que $\forall c \in \text{ShortcutSet}, c \notin \text{PathSet}$ et $\text{IsLethal}(c) = \text{false}$.
 6. Vérification qu'aucun cul-de-sac isolé ne reçoit le statut `SafeShortcut`.
 7. **Résolution sur tailles impaires :**
 8. Validation de la résolubilité sans freeze pour $N \in \{11, 13, 15, 27, 33, 49\}$.
 9. **Indépendance de présentation :**
 10. Exécution complète du serveur sans HUD ni rendu 3D.
-

Document synchronisé avec le script `cube_solveur_blender_3d.py` (branche `Blender_PY`).